

Subject: Meeting Notes - LLM Wiki vs KB Approaches
Date: Monday, May 4, 2026 at 9:26:28 PM Pacific Daylight Time
From: Harish Parameswaran
To: Sravan Sunkaranam, Rajeev Bansal
Attachments: caption_Meeting_2.txt, caption_Meeting_1.txt

Confidential - Oracle Restricted \Including External Recipients

Recording 1 - https://otube.oracle.com/media/KB%20Review/1_1tmu300e

Recording 2 - https://otube.oracle.com/media/t/1_m0cyhp28

Both zoom summaries are in below



Meeting assets for Harish P's Personal Meeting Room are ready!

[View meeting recap](#)

Meeting summary

Key takeaways

- LLM Wiki is suitable for small-scale, personal knowledge bases and specific use cases like TPM documentation and code documentation, but faces challenges with enterprise-scale scenarios
- Knowledge base systems with structured databases are better suited for large-scale operational data, incident management, and scenarios requiring efficient retrieval, concurrency, and access control
- Content parsing should be handled by LLM for contextual data, but relational/fleet data should bypass LLM parsing and be stored directly in databases
- Key differentiators include: scale and performance requirements, concurrency support, permission management, cost of LLM token usage, and deterministic vs non-deterministic data extraction
- Incident management data was identified as a strong use case to demonstrate the superiority of knowledge graph approaches over LLM Wiki

Discussed topics

LLM Wiki vs Knowledge Base for Different Use Cases

The team discussed the applicability of LLM Wiki versus structured knowledge bases across various scenarios, examining where each approach excels.

- **Details**

- Sravan: LLM Wiki works well for use cases where source content is already in wiki format, like Ankur's presentation and Sanjay's work, which predominantly use wiki content with some Jira details
- Sravan: For Zoom's project, they used LLM Wiki for a greenfield product with no legacy baggage, where Claude maintained documentation along the development process
- Rajeev: LLM Wiki doesn't work well for scenarios requiring scale and performance, such as incident management with thousands of tickets weekly
- Harish: LLM Wiki works at individual or small team level but struggles at organizational scale due to concurrency and maintenance challenges
- Sravan: TPM knowledge base could work with LLM Wiki since it likely won't exceed 10,000-20,000 pages in the lifetime of the service

- **Conclusion**

- LLM Wiki is appropriate for small-scale, structured documentation use cases like TPM wikis and code documentation
- Enterprise-scale operational data requires structured knowledge bases with proper indexing and database storage
- The format choice depends on the specific use case, data volume, and access patterns

Scale and Performance Limitations of LLM Wiki

Discussion centered on whether LLM Wiki can handle large-scale enterprise data and concurrent access patterns.

- **Details**

- Rajeev: At scale with thousands of tickets and multiple simultaneous users, LLM Wiki becomes inefficient due to token usage and lack of concurrency support
- Sravan: Challenged the team to provide concrete examples where LLM Wiki fails at scale, noting that intuition alone isn't sufficient justification
- Rajeev: Referenced Codex LLM's guidance that knowledge bases with RAG and graph are better for storing large operational data and retrieving facts efficiently
- Harish: Raised concerns about handling 93 P2T issues over a weekend and maintaining 25-30 different attributes per incident at scale
- Sravan: Acknowledged that if there are millions of documents or 100,000+ extracted contents, efficient retrieval from wiki formats becomes questionable

- **Conclusion**

- Scale is a valid concern but requires concrete proof through implementation
- Token consumption costs increase significantly with large-scale LLM Wiki implementations
- Concurrency and parallel updates are problematic in LLM Wiki systems

Content Parsing and Data Extraction Approaches

The team explored different approaches to parsing and extracting content from various data sources.

- **Details**

- Sravan: Proposed a layered approach where content creation and retrieval always go through a parser (LLM or custom), which extracts metadata, context/summary, and relationships

- Rajeev: Described the current knowledge base approach using metadata extraction, content parsing, chunking, and multiple index types including vector, lexical, document structure, and knowledge graph
- Sravan: Distinguished between data worth parsing through LLM versus relational/fleet data that should bypass LLM processing
- Rajeev: Emphasized using deterministic extraction rules rather than letting LLM decide what to extract, providing business rules upfront
- Harish: Questioned how to maintain source of truth when LLM is parsing content, noting every meeting note and document would need to be fed through LLM

• Conclusion

- Content parsing should be LLM-driven for contextual data where summarization adds value
- Fleet data and relational data should not go through LLM parsing as it provides no benefit
- Deterministic business rules should guide extraction rather than fully autonomous LLM decisions
- Every content creation must go through the parser for the system to work effectively

Knowledge Base Architecture and Index Types

Discussion of the multi-layered knowledge base architecture with different index types for different purposes.

• Details

- Rajeev: Presented the knowledge base architecture with multiple components: document index, vector index, lexical index, document structure index, knowledge graph, synonym/terminology index, and analytical indexes
- Rajeev: Explained that different domains use different retrieval methods and graphs, with query expansion for natural language queries
- Sravan: Questioned the data format used in document structure index and how property graphs are stored in Oracle database
- Harish: Clarified that Oracle stores property graphs natively using vertex tables and edge indexes as a metadata layer on relational data
- Rajeev: Described the process of finding nodes first using vector embeddings, then traversing the graph using PG (property graph) indexes

• Conclusion

- Knowledge base uses multiple specialized indexes for different data types and retrieval patterns
- Property graphs are stored in database structures that enable efficient traversal
- The architecture supports both semantic search and structured relationship queries

Incident Management as a Use Case

The team identified incident management as a strong candidate to demonstrate knowledge base advantages over LLM Wiki.

• Details

- Rajeev: Proposed incident management as a use case that doesn't fit LLM Wiki due to scale, with thousands of tickets requiring structured storage and efficient retrieval
- Harish: Noted that ADP alone had 760 patterns in one month and 93 issues in P2T over a weekend, requiring maintenance of 25-30 attributes per incident
- Sravan: Analyzed that incident data involves reading Jira tickets and log files, associating metadata about services and pods, which qualifies as operational data
- Rajeev: Suggested using incident data to demonstrate blast radius calculation, combining historical issue resolution with dependency graphs

- Harish: Emphasized that incident management needs to span domains and boundaries, requiring relationship tracking across the entire system

- **Conclusion**

- Incident management is a strong use case for demonstrating knowledge base superiority
- The use case involves operational data at scale with complex relationships
- Blast radius calculation requires both historical data and real-time dependency graphs

Relational Data vs Contextual Data

Discussion on distinguishing between different types of data and appropriate storage mechanisms.

- **Details**

- Sravan: Distinguished between relational data (fleet data with instances), semantic data (specs and definitions), and contextual data (documents requiring summarization)
- Sravan: Argued that fleet data like 20,000 pods with 20 properties each should not go through LLM parsing as there's nothing to summarize
- Rajeev: Agreed that fleet data is not suitable for LLM Wiki, but incident data is operational data that needs structured storage
- Sravan: Clarified that schema-defined data with instances is always a good qualifier for relational databases, not wikis
- Harish: Noted that NL2SQL use cases clearly won't work on LLM Wiki due to the relational nature of the data

- **Conclusion**

- Relational/fleet data should be stored in databases without LLM parsing
- Contextual data from documents benefits from LLM parsing and summarization
- Operational data like incidents falls between these categories and requires careful consideration

Permission Management and Access Control

The team discussed enterprise requirements for permission management that LLM Wiki cannot easily support.

- **Details**

- Sravan: Raised permission management as a critical issue, noting that databases provide roles, policies, and access control that wikis lack
- Rajeev: Agreed that access control must be enforced at the data access layer, and once data is extracted and transformed, it's very hard to filter
- Sravan: Pointed out that LLM maintains knowledge base and can access everything, but has no way to know what individual consumers should see
- Rajeev: Noted that in the personal knowledge base model, users already have access to documents, but enterprise scenarios require different access models for large organizations
- Sravan: Emphasized that wiki management requires a system like Confluence to provide enterprise bells and whistles including permission management

- **Conclusion**

- Permission management is a significant advantage of database-backed knowledge bases
- LLM Wiki lacks built-in access control mechanisms
- Enterprise scenarios require sophisticated permission models that wikis cannot easily provide

Cost and Efficiency of LLM Token Usage

Discussion on the cost implications of using LLM for parsing and retrieval at scale.

- **Details**

- Rajeev: Highlighted that LLM Wiki requires tokens for both extraction and traversal, with costs multiplying based on workflow iterations
- Sravan: Acknowledged that if LLM does a better job of extracting context and all content is for LLM retrieval, the ingestion cost may be worth it
- Harish: Joked that the organization would become OpenAI's biggest customer if every meeting note and document goes through LLM
- Rajeev: Compared to deterministic orchestration where retrieval happens without LLM cost, only using LLM for final parsing
- Sravan: Argued that incurring LLM cost for traversal is not worth it when better retrieval mechanisms exist, like SQL queries

- **Conclusion**

- LLM token costs are significant at enterprise scale
- Extraction costs may be justified, but traversal costs should be minimized
- Deterministic retrieval methods are more cost-effective than LLM-driven traversal

Storage Format and Retrieval Mechanisms

The team debated whether the storage format (wiki vs database) matters as much as the parsing and retrieval approach.

- **Details**

- Sravan: Argued that the key is LLM parsing and management of extracted content, while physical storage format is secondary
- Sravan: Noted that LLM Wiki documentation uses wiki as one storage method, but content could be stored in databases or object stores
- Rajeev: Emphasized that efficient storage and retrieval are critical, which is why the knowledge base uses multiple index types
- Sravan: Pointed out that the main catch is LLMs are efficient at reading markdown files but not at writing SQL queries
- Harish: Questioned whether LLM Wiki stored in a database is still LLM Wiki or just using database for storage

- **Conclusion**

- The parsing approach (LLM-driven vs deterministic) is more important than storage format
- Storage format should be chosen based on retrieval efficiency requirements
- LLMs are better at reading markdown than generating SQL, which influences format choice

Relationship Management and Graph Traversal

Discussion on how different approaches handle relationships between data entities.

- **Details**

- Sravan: Questioned how LLM Wiki would maintain relationships as new content comes in without business context
- Rajeev: Explained that knowledge graph uses property graphs with nodes and edges, finding nodes via vector embeddings then traversing relationships
- Sravan: Suggested that if LLM understands business structure (like ADP's resources and operations), it can organize content and maintain relationships
- Rajeev: Noted that LLM-driven graph traversal requires LLM to decide which next node to visit at each step, incurring costs

- Harish: Asked about maintaining relationships across 446 wikis in a confluence space and tracking evolution over 46 versions

- **Conclusion**

- Relationship management requires either business rules or LLM understanding of domain structure
- Graph traversal via LLM is costly compared to database-backed graph queries
- Complex relationship tracking across large wiki sets is challenging for LLM Wiki

Existing Data Migration and Maintenance

The team discussed challenges of ingesting existing data and maintaining knowledge bases over time.

- **Details**

- Sravan: Identified large existing data sets as a challenge, but argued the cost is justified if LLM does better extraction and retrieval is LLM-targeted
- Sravan: Noted that LLM Wiki's advantage is that maintenance happens automatically as LLM accesses content, rather than running batch workflows
- Rajeev: Pointed out that LLM Wiki requires content to be accessed through LLM for updates, so unchanged source content means stale knowledge
- Harish: Questioned how to feed 450+ wikis with full version history into LLM for context building
- Sravan: Suggested feeding all conference pages once to extract initial content, then maintaining through ongoing access

- **Conclusion**

- Initial ingestion of existing data is costly but potentially justified
- LLM Wiki maintenance is event-driven (on access) rather than batch-driven
- Stale data is a risk if content isn't regularly accessed through LLM

Technical Issues and Database Connectivity

Brief discussion of technical problems Rajeev encountered during the meeting.

- **Details**

- Rajeev: Reported that database connections were working but SQL queries were timing out and sessions were stuck
- Harish: Suggested checking for DigiCert certificate issues and trying SQL Developer
- Rajeev: Noted that search functionality was broken on Mac after restart
- Harish: Attempted to connect using the same credentials to verify if the issue was widespread

- **Conclusion**

- Database connectivity issues prevented live demonstration during the meeting
- The team planned to troubleshoot after the meeting

Action items

- **Rajeev**

- ☐ Build both LLM Wiki and Knowledge Base implementations in parallel for incident management use case to compare performance and scalability
- ☐ Create a demonstration showing where LLM Wiki breaks at scale with specific data volume thresholds
- ☐ Define business rules and ontology for incident data extraction and storage

- ☐ Resolve database connectivity and SQL query timeout issues

- **Harish**

- ☐ Test LLM Wiki's ability to process version history across multiple confluence pages
- ☐ Prepare incident data examples showing blast radius calculation requirements

- **Sravan**

- ☐ Document the layered architecture approach distinguishing content parsing, storage format, and retrieval mechanisms
- ☐ Define clear criteria for when data is "worth parsing" through LLM vs direct database storage

- **Team**

- ☐ Develop concrete examples demonstrating LLM Wiki limitations at enterprise scale
- ☐ Create a framework for evaluating use cases against LLM Wiki vs Knowledge Base criteria
- ☐ Schedule follow-up meeting at 5:30 PM to continue discussion

[View in Zoom](#)

MEETING 2 at 830 PM

Meeting summary

Key takeaways

- The team is developing a comprehensive knowledge base framework that recognizes different data types require different storage and retrieval approaches, not a one-size-fits-all database solution
- LLM's role differs between ingestion and retrieval: LLM is valuable in the ingestion pipeline for certain data types but often inefficient for retrieval, where vector search proves more effective
- Three main data types identified: operational incident data using vector DB, fleet data using SQL, and code requiring specialized indexing like LLM wiki or AST
- Two critical unsolved problems: how to efficiently store and retrieve LLM wikis for remote agent access, and how to handle product management/TPM documentation
- The solution involves a context builder agent that navigates through a shim layer of LLM wikis to determine which data sources to access based on user queries

Discussed topics

Meeting logistics and scheduling

The team discussed rescheduling a conflicting fleet inventory call to accommodate the current discussion.

- **Details**

- Harish: Noted a 9 o'clock fleet inventory call with Saurabh and Smitha, suggesting 30 minutes available
- Sravan: Requested moving the call to tomorrow morning at 8:30 or 9 o'clock to complete the current conversation
- Harish: Confirmed rescheduling to 9 o'clock tomorrow morning

- **Conclusion**

- Fleet inventory call rescheduled to tomorrow morning at 9 o'clock

Knowledge base framework architecture overview

The team established the foundational approach for building a knowledge base that accommodates multiple data types with different storage and retrieval mechanisms.

- **Details**

- Sravan: Explained that ingestion and retrieval should happen through the same parser, which could be LLM or other tools, except for fleet data and code
- Sravan: Identified fleet data and code as special qualifiers that don't fit the standard LLM wiki approach
- Sravan: Emphasized the need to differentiate between LLM's role in ingestion versus retrieval pipelines
- Sravan: Stated the key is figuring out for each data type what is the best way to store and retrieve

- **Conclusion**

- Different data types require different storage and retrieval strategies
- LLM's role must be evaluated separately for ingestion and retrieval phases
- Fleet data and code are exceptions requiring specialized handling

Operational incident data and Aira journey

The team reviewed their experience building the operational knowledge base for Aira, which informed their current approach.

- **Details**

- Sravan: Described starting with Aira operational knowledge base and learning that LLM in the ingestion pipeline helped extract reusable context
- Sravan: Explained they summarized incidents with LLM and indexed raw data, which significantly helped
- Sravan: Noted that for retrieval, vector search proved more efficient than using LLM
- Sravan: Mentioned they tried storing raw data, logs, and cataloging Jira tickets, but none helped as much as their current approach
- Rajeev: Confirmed vector search works better than lexical search for this use case

- **Conclusion**

- LLM is valuable in the ingestion pipeline for summarizing and extracting context
- Vector search is more efficient than LLM for retrieval in operational incident data
- Chunking and storing summarized content with raw data indexing proved most effective

Code storage and retrieval approach

The team discussed Som's work on LLM wiki for code and explored how to make code accessible to remote agents.

- **Details**

- Rajeev: Explained Som created an LLM wiki that stores code structure rather than vectorizing everything, allowing efficient navigation to specific folders
- Sravan: Questioned how remote agents like Aira would access the codebase
- Rajeev: Proposed that when Aira needs to fix code, it could spin up a VM, download code using SCM, create the index using the skill, fix the issue, push a branch, and create a PR

- Sravan: Acknowledged this as one approach but indicated more exploration needed for code handling
- Sravan: Suggested creating a shim layer of LLM wikis for initial context about services, features, and architecture

- **Conclusion**

- LLM wiki for code structure is a viable indexing approach instead of AST
- Remote agent access to code requires further investigation
- A shim layer can provide high-level navigation context for code

Fleet data handling

The team confirmed their approach for fleet data storage and retrieval.

- **Details**

- Sravan: Stated fleet data should use Sentinel or similar SQL-based solutions
- Sravan: Argued there's no point passing 20,000 pods data to LLM to create a wiki
- Sravan: Emphasized SQL is the right tool for fleet data and this doesn't need further justification

- **Conclusion**

- Fleet data will use SQL-based storage like Sentinel
- LLM wiki approach is inappropriate for large-scale fleet data
- This approach is straightforward and doesn't require additional examples

Product management and TPM documentation

The team identified product management and TPM data as likely candidates for LLM wiki but recognized storage challenges.

- **Details**

- Sravan: Suggested product management and TPM design documents likely qualify for LLM wiki since they're already in Wikipedia format
- Sravan: Noted if maintained automatically by LLM, it would be even better
- Sravan: Pointed out that TPM agents producing weekly summaries and status reports would naturally fit wiki page format
- Sravan: Emphasized the need to figure out how to efficiently store and retrieve hundreds or thousands of LLM wikis
- Sravan: Mentioned discussing with Claude, which initially suggested vector database but then agreed it's a bad choice for LLM wikis

- **Conclusion**

- Product management and TPM documentation are good candidates for LLM wiki
- The storage and retrieval mechanism for LLM wikis remains an open problem requiring investigation
- Vector database is not the right solution for storing LLM wikis

LLM wiki storage investigation

The team explored options for storing LLM wikis efficiently, considering various approaches.

- **Details**

- Sravan: Stated LLM wikis should be managed through code for revision control, update tracking, and hooks
- Sravan: Raised the challenge of making code-based wikis available to centralized remote agents

- Sravan: Mentioned Claude suggested multiple options including graph database or other alternatives
- Rajeev: Agreed to investigate and figure out the best storage approach

- **Conclusion**

- LLM wikis should be stored in code repositories for version control
- The challenge is making code-based wikis accessible to remote agents
- This is a critical open problem requiring team investigation

Context builder and retrieval orchestration

The team discussed how agents would retrieve context from multiple data sources through an orchestration layer.

- **Details**

- Sravan: Proposed a context builder or retrieval agent that navigates through a shim layer of LLM wikis
- Sravan: Provided detailed example of a complex query about customer-initiated P2Ps on large databases before/after FA maintenance
- Sravan: Explained the agent would first understand concepts through LLM wiki, then access fleet data, code, or other sources as needed
- Harish: Asked whether the agent would go through all orchestration steps or directly to the database
- Sravan: Suggested providing all available data sources and letting the LLM figure out what to access, similar to how a human would approach it
- Sravan: Emphasized the need for a context packet builder that can access all tools and data sources

- **Conclusion**

- A context builder agent will navigate the shim layer to determine required data sources
- The agent should be given information about all available data sources and their contents
- LLM should determine the optimal path based on the query, similar to human reasoning

Knowledge generation for different data types

The team outlined what knowledge needs to be extracted from each data source type.

- **Details**

- Sravan: Stated code knowledge generation involves creating LLM wikis
- Sravan: Explained fleet data requires creating semantics, possibly as a table or another LLM shim layer
- Sravan: Noted operational incident data already has defined extraction: root cause, logs, impact, resources
- Sravan: Identified that TPM and product management knowledge extraction is still an individual exercise not yet solved
- Sravan: Emphasized the knowledge base framework won't solve what to extract, but will provide the infrastructure to store and retrieve it

- **Conclusion**

- Each data type requires specific knowledge extraction approaches
- Operational incident knowledge extraction is already defined
- TPM and product management knowledge extraction remains to be determined
- The framework provides infrastructure, not content definition

Jira roadmap data handling

The team briefly discussed how to handle Jira tickets related to product roadmaps.

- **Details**

- Sravan: Mentioned Jira tickets for roadmap need to be figured out
- Sravan: Suggested possibly creating service-specific roadmaps
- Sravan: Indicated this is another area requiring investigation for knowledge management

- **Conclusion**

- Jira roadmap data handling approach needs to be determined
- Service-specific roadmaps may be the solution
- This remains an open item for investigation

Database versus multi-source approach

The team clarified that the solution is not a single database but a multi-source system.

- **Details**

- Sravan: Corrected the assumption that everything should be in a database
- Sravan: Emphasized code will never be kept in a database
- Rajeev: Acknowledged the distinction between database storage and procedural memory/LLM wiki
- Sravan: Stated database has a role for certain things but not everything

- **Conclusion**

- The solution is not a unified database but a multi-source system
- Different data types will reside in appropriate storage systems
- Code and LLM wikis will not be in traditional databases

Access control and permissions

The team acknowledged permissions as a future layer to be addressed after establishing the core framework.

- **Details**

- Harish: Raised the question of permission models
- Sravan: Confirmed this is the next level of discussion after establishing the initial framework
- Sravan: Stated access control at the data level and other details are layers to build on top

- **Conclusion**

- Permission and access control will be addressed after core framework is established
- This is recognized as an important but secondary concern
- Data-level access control is a layer to be built on top of the base system

Presentation strategy and narrative

The team discussed how to present their approach and findings.

- **Details**

- Harish: Suggested focusing on incident management and fleet data use cases where they know the approach works
- Sravan: Clarified fleet data argument is straightforward and doesn't need extensive examples

- Sravan: Explained the operational data case needs to clarify why LLM is in ingestion but not retrieval
- Harish: Proposed breaking the narrative into three parts: where it works, where it doesn't work, and undetermined cases
- Sravan: Emphasized the storyline should start with a use case, acknowledge multiple data sources, and explain different requirements

- **Conclusion**

- Presentation will follow a narrative starting from use cases to solution
- Focus on explaining why different approaches work for different data types
- Simplify the complex diagrams and focus on clear storyline

Documentation and next steps

The team outlined immediate action items for documenting their discussion and continuing research.

- **Details**

- Sravan: Requested using Codex to write up the discussion with pictures
- Harish: Confirmed they would summarize and determine the narrative structure
- Sravan: Identified two main research problems: LLM wiki storage/retrieval and code accessibility for remote agents
- Rajeev: Agreed to rework the diagram to reflect the simplified approach
- Sravan: Suggested using the meeting transcript to generate a document
- Harish: Confirmed he would send both meeting transcripts

- **Conclusion**

- Team will use AI tools to generate documentation from meeting transcript
- Diagram needs to be simplified to convey the core concepts
- Two critical research areas identified for immediate investigation

FA semantic data use case

The team briefly discussed the semantic data work being done with Dave's team.

- **Details**

- Sravan: Asked about the semantic data story from Dave
- Rajeev: Explained Dave is doing a POC using their code for FA schema-related constraints
- Harish: Clarified the use case is for developers adding new objects or rules to understand potential impact
- Rajeev: Stated they're using graph to store object relationships and rules to detect violations
- Sravan: Noted this follows the standard workflow: determine if LLM is needed for ingestion/retrieval, decide storage format, then determine storage location
- Rajeev: Mentioned a call scheduled for tomorrow at 4 o'clock

- **Conclusion**

- FA semantic data is being explored using graph storage
- The use case involves detecting constraint violations when adding new objects
- Storage location in UDAP or standalone system is still undetermined
- This follows the same evaluation framework as other data types

Timeline and meeting coordination

The team discussed coordination with other meetings and presentation timelines.

- **Details**

- Sravan: Mentioned a meeting tomorrow where 100 people might ask questions
- Sravan: Stated the current knowledge base topic is too complex to cover in that forum
- Rajeev: Suggested introducing UDAP platform in general first, then knowledge base later
- Sravan: Disagreed, noting they've already planted seeds about the concept and shouldn't delay
- Sravan: Requested the document be ready by tomorrow morning for discussion

- **Conclusion**

- Knowledge base presentation shouldn't wait for full UDAP introduction
- Document needs to be ready by tomorrow morning
- The topic is too complex for the large forum meeting

Action items

- **Sravan, Rajeev, Harish**

- ☐ Research and determine the best way to store and retrieve LLM wikis for remote agent access
- ☐ Figure out how to make code with LLM wiki layer accessible to remote agents

- **Rajeev, Harish**

- ☐ Rework the knowledge base diagram to simplify and clearly convey the discussed concepts
- ☐ Use Codex or similar tool to generate a comprehensive document from the meeting transcript with diagrams
- ☐ Complete documentation by tomorrow morning for team discussion

- **Harish**

- ☐ Send transcripts from both the previous meeting and current meeting tonight

- **Team**

- ☐ Review and iterate on the generated document tomorrow morning
- ☐ Determine the best approach for storing product management and TPM documentation
- ☐ Define what knowledge to extract from TPM and product management documents

[View in Zoom](#)